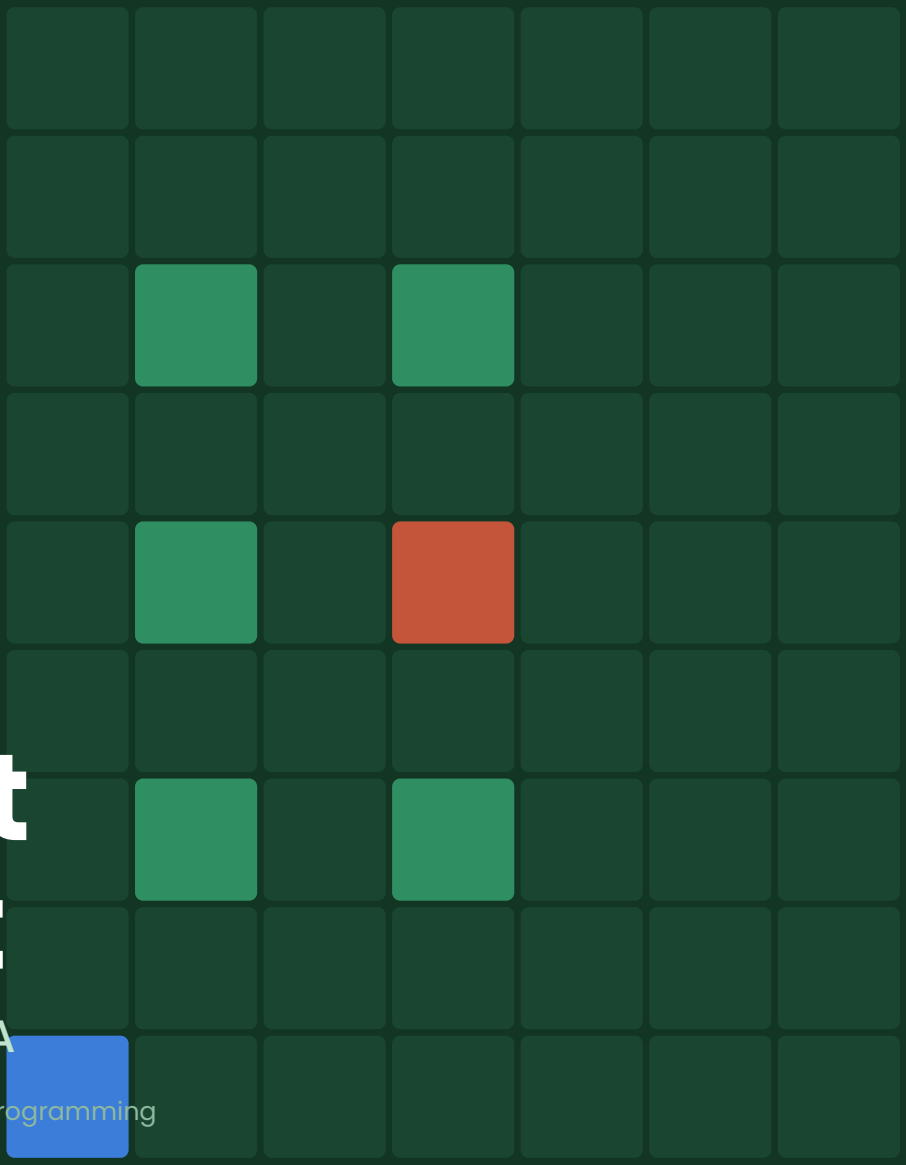


Project Report

Smart Orchard WA

COMP5005 · Fundamentals of Programming

NAME
Nicolas Belles

STUDENT NUMBER
24210773

DATE
24/10/2026

Curtin University

Overview

Smart Orchard WA is a discrete event, object-oriented simulation program developed to simulate an automatic harvesting system for orchards. Coordinating the movements of autonomous agents which explore a grid-based map to find, pick, and bring ripe crops back to their home depots efficiently is the aim of the program.

The features that have been implemented are: A detailed Finite State Machine (FSM) for each robot behaviour, Dynamic procedural map generation, Thorough pre-execution topological map validation, and best collision-free route generation using Breadth-First Search (BFS) with Manhattan Distance pruning. The system can be used for both real-time visualization and for batch processing for performance analysis.

User Guide

The simulation runs from the `src/` directory using the command line.

Prerequisites

Make sure that Python 3.8+ is installed. Use your terminal to install the following packages:

```
$ pip install pandas matplotlib seaborn notebook
```

Interactive Mode (Live Visual Subplots)

Run the live simulation:

```
$ python smartfarm.py -i
```

The user will be asked the size of the grid, the number of robots, and the number of timesteps. `Enter` is used to use the safe default parameters. The spatial grid, inventory tracking and throughput efficiency will be displayed graphically on a live dashboard.

Batch Mode & Generating Data for Analysis

To run the showcase analysis, the three predefined benchmark scenarios need to be run first. The simulations are processed silently and the performance metrics needed and the plots in PNG are exported to the `results/` directory:

- **Scenario 1:** This script generates a scenario for the most basic use case, in which the farmer is located in the center of the farm, and the field is rectangular.

  ```
  $ python smartfarm.py -f ../data/map_s1.csv -p ../data/params_s1.csv
  ```

- The second scenario contemplates the case where we double the amounts of robots in the simulation:

  ```
  $ python smartfarm.py -f ../data/map_s2.csv -p ../data/params_s2.csv
  ```

- Escenario 3 is similar to the baseline, but with a delay ready_timestep of 15 steps:

  ```
  $ python smartfarm.py -f ../data/map_s3.csv -p ../data/params_s3.csv
  ```

Showcase Analysis Notebook

After the batch scenarios have been correctly populated into the `results/` folder, open the `notebooks/showcase_analysis.ipynb` file in Jupyter Notebook or VS Code. Execute all of the cells one at a time to see the comparative visualizations and the quantitative performance summary as well as the vectorized Conservation Law unit tests.

Robustness and Error Testing

The `test/` directory includes some intentional edge cases that showcase the strict error handling and topological validation of the system. These can be tested by running:

- Invalid Map (Unreachable Crops):

  ```
  $ python smartfarm.py -f ../tests/invalid_map.csv -p ../data/params_s1.csv
  ```

  (Safely aborts and raises a ValueError stating the missing driveable tracks).

- Raises ValueError on invalid parameters (negative values):

  ```
  $ python smartfarm.py -f ../data/map_s1.csv -p ../tests/invalid_params.csv
  ```

  (Safely aborts and raises a ValueError on invalid parameters (negative values)).

Project Structure

This section describes how the project repository is organized both logically and physically. A consistent directory organization provides robustness and reproducibility by clearly partitioning input data sets, source scripts, outputs from executing code, and analytical notebooks. The directory hierarchy is shown below (Figure 1) and any automated check can be immediately executed from the project's base folder:

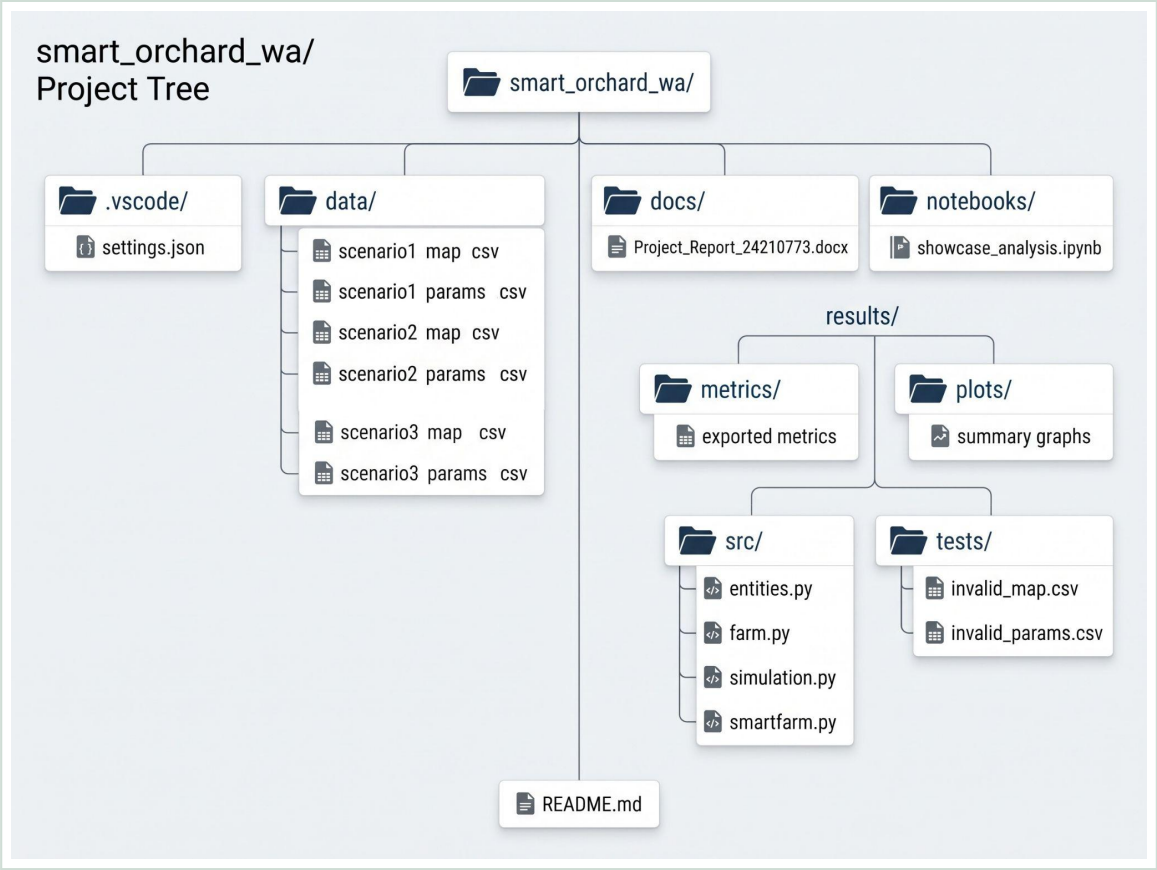


Fig 1. Image showing the logical structure of the Smart Orchard WA project.

Traceability Matrix

Table giving overview of features, and the implementation and testing of your code.

- **Feature** - numbered for easy referencing
- **Code reference** – reference to files/classes/methods or snippets of code only, do not put the whole program in the report OR “Not Implemented”
- **Test reference** – test code or describe how you tested your feature, N/A if not implemented
- **Status** – P = passed tests, S = skipped, F = failed, or N/A
- **Date Completed** – date or “Ongoing” or N/A if not implemented

Feature	Code Reference	Test Input	Expected Outcome	Actual Outcome	Status	Date Completed
1.0 Core Entities & Behaviors						
1.1 Robot State Machine	`Robot` class (`step_change`)	`showcase_analysis.ipynb` (Unit test suite)	correct transition between the states WAITING, TRAVELLING, HARVESTING, RETURNING, UNLOADING.	Correct state transitions logged without locking.	**P**	14/10/26
1.2 Target Selection & Pruning	`choose_target()` in `Robot` class	Boundary: Multiple crops at equal linear distance.	The tie-breaking rule selects a smaller row/col index deterministically.	Deterministic selection verified via unit tests.	**P**	27/10/26
1.3 Crop Maturation & Sensors	`CropBlock` class in `entities.py` (`sensor_reading`)	Edge Case (No Ripe Crops): Simulation run with `ready_timestep = 15` (Scenario 3 configuration)	Robots remain in WAITING state, polling `sensor_reading()` returning 0 until timestep reaches 15..	Agents stayed idle at depots until maturation timestep, successfully handling temporal constraints.	**P**	17/10/26
2.0 Terrain & Navigation						
2.1 CSV Map Parsing	`load_map()` in `Farm` class	Normal Case: Loading well-formed rectangular layout files (`scenario1_map.csv`).	Grid matrix parsed correctly; terrain symbols (T, C, D, X) mapped without shape distortion.	Maps successfully loaded and validated against parameter configuration files.	**P**	26/10/26
2.2 Shortest Route (BFS)	`shortest_route()` in `Farm` class	Concurrency Case: Multi-agent scaling (`robot_count = 4`) targeting shared, finite crop pools.	Dynamic retargeting prevents duplicate harvesting or negative inventory states when a crate is exhausted.	Conservation Law verified at every step; no duplicate picks or negative inventory occurred.	**P**	24/10/26
2.3 Reachability Validation	`validate()` & `get_all_reachable_cells()` in `Farm`	Invalid Case (Unreachable Crops): Executing map with an isolated crop (`tests/invalid_map.csv`).	System intercepts topological flaw before execution and raises a descriptive ValueError.	Raised ValueError: Crop block at (2, 2) has no adjacent driveable track.	**P**	21/10/26
3.0 Simulation & Analytics						

Feature	Code Reference	Test Input	Expected Outcome	Actual Outcome	Status	Date Completed
3.1 Discrete-Event Loop	`step_change()` in `Simulation` class	Normal Case: Temporal execution loop across configured timesteps.	Updates sensors, actions, and statistics sequentially while maintaining invariant balances.	Invariant verified: Initial Crates = Unharvested + Carried + Delivered at every timestep.	**P**	23/10/26
3.2 Live Rendering & Export	Export `_render_frame()` & `save_results()`	Normal Case: Batch execution completion across predefined scenarios.	High-resolution summary charts (.png) and metrics logs (.csv) saved to `results/` folder.	Artifacts successfully exported and validated in analytical pipeline.	**P**	21/10/26
3.3 Input Error Handling	`get_user_input()` & `load_params_csv()`	Invalid Case (Negative Parameters): Parameter file containing `robot_count = -2` (`tests/invalid_params.csv`).	System rejects negative values, prints a clear warning, and aborts batch execution safely.	Raised ValueError: robot_count must be positive.	**P**	23/10/26

Discussion

The program is heavily based on Object-Oriented principles, with an emphasis on encapsulation and modularity. The `Simulation` class is the system coordinator and controls the temporal loop and calls invariant checks (Conservation Law). The `Farm` class groups the terrain matrix and `CropBlock` objects, and the `Robot` class is a model that implements decentralized decisions by using local sensor data.

To visually illustrate how the Smart Orchard WA simulation is structured and how its components interact, a UML Class Diagram is presented below.

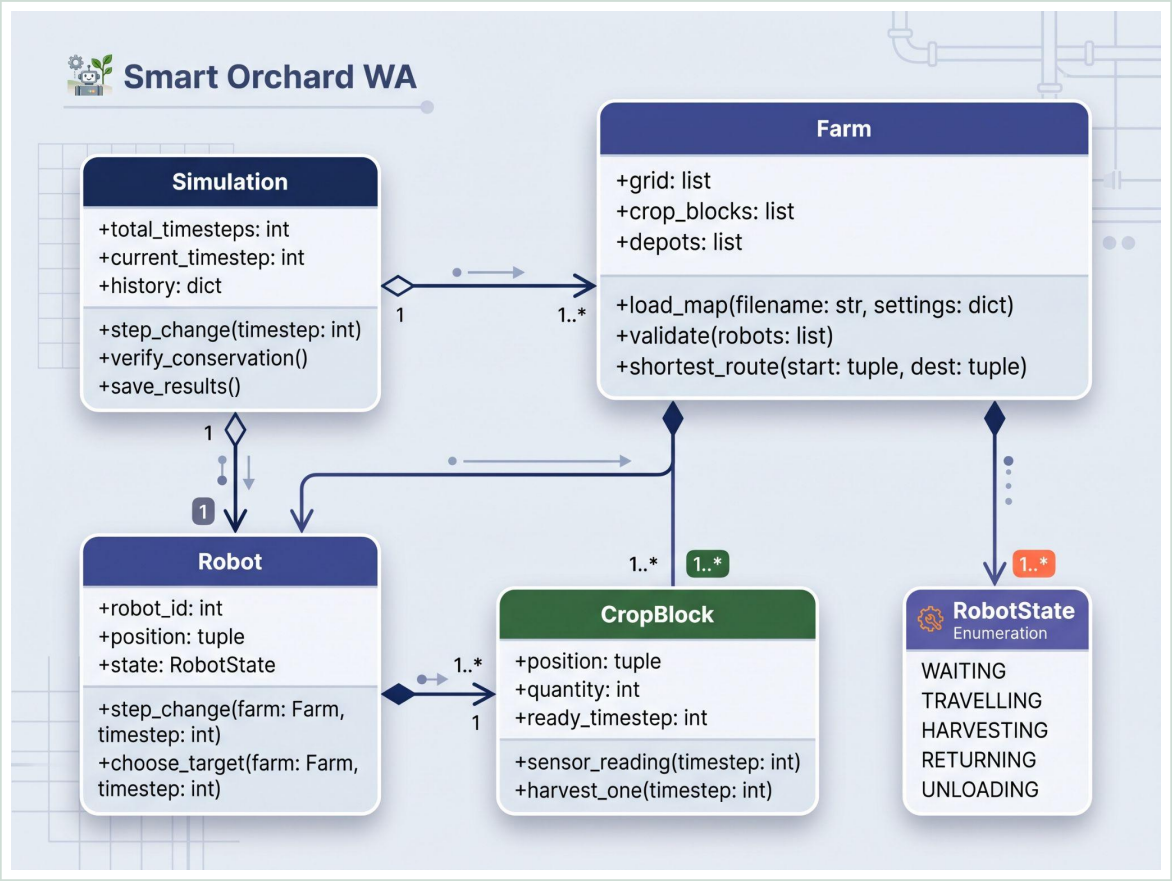


Fig 2. UML diagram of the Smart Orchard project.

This diagram highlights the encapsulation and structural relationships between the object-oriented modules. The main Simulation class acts as the system's discrete-event coordinator, maintaining Composition relationships (1-to-many) with the robotic agents (Robot) and the terrain manager (Farm). The Farm class, in turn, contains and manages the spatial positioning of the various crop locations (CropBlock). Robotic units operate through a strictly managed State pattern via the RobotState Enum, while interacting with both the Farm for obstacle-free pathfinding and individual CropBlocks for harvesting transactions.

Algorithmic Implementations

1 The Map Reachability Validation (Flood Fill)

To keep things robust—and avoid disasters at runtime—the simulation runs a topological check up front with `Farm.validate` .

TECHNICAL MECHANISM

before anything starts moving, the system kicks off a Flood Fill using BFS, starting from all active depots. It basically colors in the terrain matrix, marking every cell (T and D) that you can actually reach.

IMPLEMENTATION BENEFIT

The validator knows exactly which cells are connected. It checks every CropBlock's neighboring harvest cells against this set. If it finds a crop that's cut off from everything, or the depots aren't set up right, it stops the process right there. You get a clear ValueError, and nothing moves forward; not even the agents. So there's no risk of agents getting stuck in an endless loop or gumming up the simulation with deadlocks.

2 Shortest Route & Manhattan Pruning

To prevent severe computational overhead when evaluating multiple destination nodes across complex layouts, the target selection pipeline (`Robot.choose_target`) incorporates a Manhattan Distance heuristic bounding step.

TECHNICAL MECHANISM

Calculating a Breadth-First Search (BFS) path for every active crop block yields a time complexity proportional to graph size (O(V + E)). By evaluating the absolute grid offset ($\Delta x + \Delta y$) prior to graph traversal, the system establishes a strict mathematical lower bound (since physical obstacles and turns can only lengthen, never shorten, a path).

IMPLEMENTATION BENEFIT

If a candidate crop's Manhattan distance exceeds the current best known shortest path (`shortest_distance`), the expensive BFS calculation is skipped instantly. This drastically optimizes multi-agent decision cycles in scaled simulation environments.

```
95      # Evaluate all available crop blocks based on sensor readings
96      for crop in farm.crop_blocks:
97          if crop.sensor_reading(timestep) > 0:
98              adjacent_tracks = farm.get_adjacent_harvest_cells(
99                  crop.position)
100
101             for cell in adjacent_tracks:
102                 # Manhattan Distance Pruning
103                 # The absolute grid distance without obstacles. The real BFS path
104                 # can NEVER be shorter than the Manhattan distance.
105                 manhattan_dist = abs(
106                     self.position[0] - cell[0]) + abs(self.position[1] - cell[1])
107
108                 # Skip expensive BFS if even the perfect path cannot beat the current best
109                 if manhattan_dist > shortest_distance:
110                     continue
111
112                 # If pruning allows it, calculate the actual valid driveable path (BFS)
113                 path = farm.shortest_route(self.position, cell)
114
115                 if path is not None:
116                     dist = len(path)
117
118                     if dist < shortest_distance:
119                         shortest_distance = dist
120                         best_crop = crop
121                         best_harvest_cell = cell
122                         best_path = path
123                     elif dist == shortest_distance and best_crop is not None:
124                         # Tie-breaking rule using direct tuple comparison
125                         if crop.position < best_crop.position:
126                             best_crop = crop
127                             best_harvest_cell = cell
128                             best_path = path
```

Fig 3. Part of `choose_target()` implementing Manhattan pruning and BFS routing.

3 Concurrency

To handle resource competition between robots without resorting to messy global locks or mutexes, we built a decentralized, sensor-based system.

TECHNICAL MECHANISM

Each robot runs on its own, never locking crop blocks for everyone. Instead, when it needs a resource, it just asks

the local CropBlock for real-time inventory using `CropBlock.sensor_reading(timestep)`.

IMPLEMENTATION BENEFIT

What happens when two robots go after the same crate? Say one is already en route, but another robot snags the last crate just before it arrives. The travelling robot checks the crate's status during its next evaluation. If it finds nothing left, it instantly calls `choose_target()` to find a new job or heads back to its depot. With this setup, there's no chance for duplicate harvesting, and we always follow the rule: no crate gets counted twice.

Showcase

A controlled experimental design framework was created to extensively test the operational features, scalability and resilience of the Smart Orchard WA simulation model. The comparative analysis examines the effect of changing the number of agents (multi-agent scaling) and the time taken for them to get mature (ripeness delays) on harvesting efficiency and throughput saturation. The layout of the maps and the parameter profiles for all the showcase experiments were standardized, and all experiments were executed automatically in batches, with the statistical aggregation of the data performed automatically using the Jupyter analysis pipeline (`showcase_analysis.ipynb`), to ensure full reproducibility. Important metrics like peak throughput, delivery rate decay and inventory conservation are constantly monitored over defined simulation horizons.

To ensure absolute reproducibility across all test runs, the following standardized configuration parameters and experimental bounds were applied:

- **Random Seed:** A fixed pseudorandom seed (seed = 42) was utilized during procedural map generation to maintain identical spatial layouts and obstacle placement across baseline and comparative scenarios.
- **Simulation Horizons (Timesteps):** Scenarios 1 and 2 were executed across a horizon of 60 timesteps (`timesteps = 60`), which provides ample duration to achieve complete harvest clearance under standard baseline conditions. Scenario 3 was executed across an extended horizon of 100 timesteps (`timesteps = 100`) to accommodate the delayed readiness schedule.
- **Simplified Farm Assumptions:** In accordance with the teaching scope of the assignment, the simulation operates under simplified pedagogical assumptions: sensors provide instantaneous and deterministic ripeness status, physical battery degradation and recharging overhead are omitted, and routing relies on static pre-calculated shortest paths (BFS) without dynamic cell-to-cell collision avoidance maneuvers among robots.

All showcase experiments were executed utilizing standardized map layouts and parameter profiles stored in the `data/` directory (`scenario1_map.csv`, `scenario2_map.csv`, `scenario3_map.csv`), with automated batch execution exporting metrics directly for subsequent statistical aggregation via the Jupyter analysis pipeline (`notebooks/showcase_analysis.ipynb`). Important operational metrics—including peak delivery throughput, rate decay, and inventory conservation balances—are systematically tracked and contrasted below.

Scenario 1

Baseline Setup: 2 robots, standard maturity ($t = 0$). Input: `map_s1.csv`, `params_s1.csv`.

Results: The system successfully harvested all 12 crates. The peak throughput reached 0.5 crates/step early in the simulation, settling at a final throughput of 0.20 crates/step. This demonstrates a stable baseline where the robotic workforce adequately matches the harvest load.

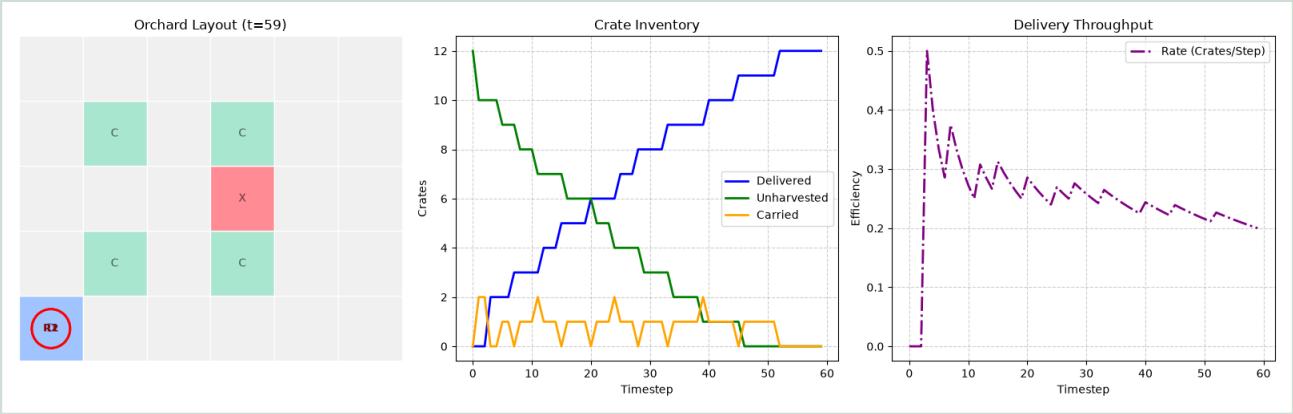


Figure 4. Results of the baseline scenario showing the crate movement over time, the efficiency of the model and the final layout.

Scenario 2

Multi-Robot Scaling Setup: 4 robots, standard maturity (t = 0). Input: `map_s2.csv` , `params_s2.csv` .

Results: Doubling the robotic workforce increased the peak throughput significantly to 0.75 crates/step. The orchard was cleared much faster, dropping the final throughput metric to 0.12 crates/step due to idle time at the end of the simulation. This highlights the system's ability to seamlessly scale and handle concurrency via dynamic retargeting without collisions or duplicate harvests.

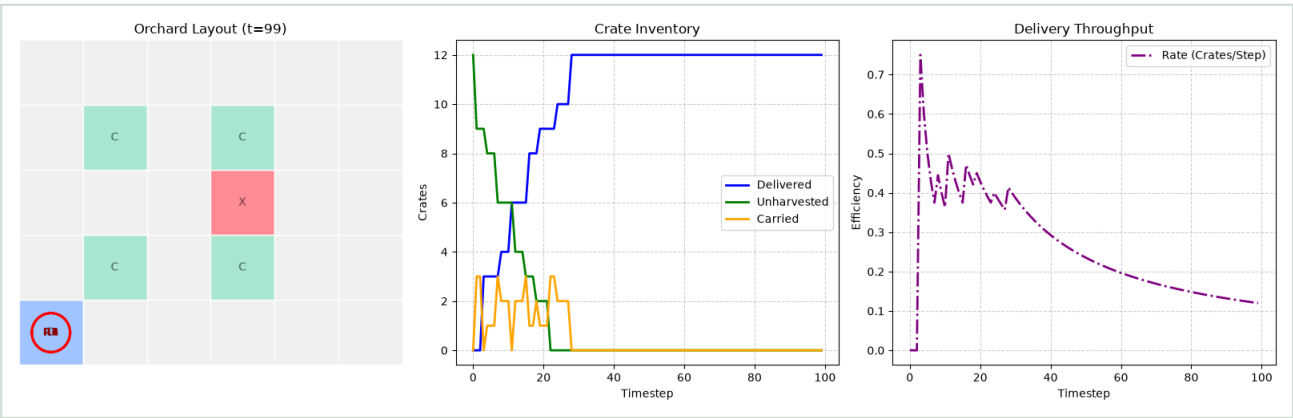


Figure 5. Results of scenario 2 simulation showing the crate movement over time, the efficiency of the model and the final layout.

Scenario 3

Delayed Readiness Setup: 2 robots, crop readiness delayed to t = 15. Input: `map_s3.csv` , `params_s3.csv` .

Results: Throughput remained at 0.0 until t = 15. Upon maturation, the peak throughput reached 0.1837 crates/step. The robots correctly remained in the WAITING state, polling sensor_reading() until availability was detected, proving the FSM accurately handles temporal constraints.

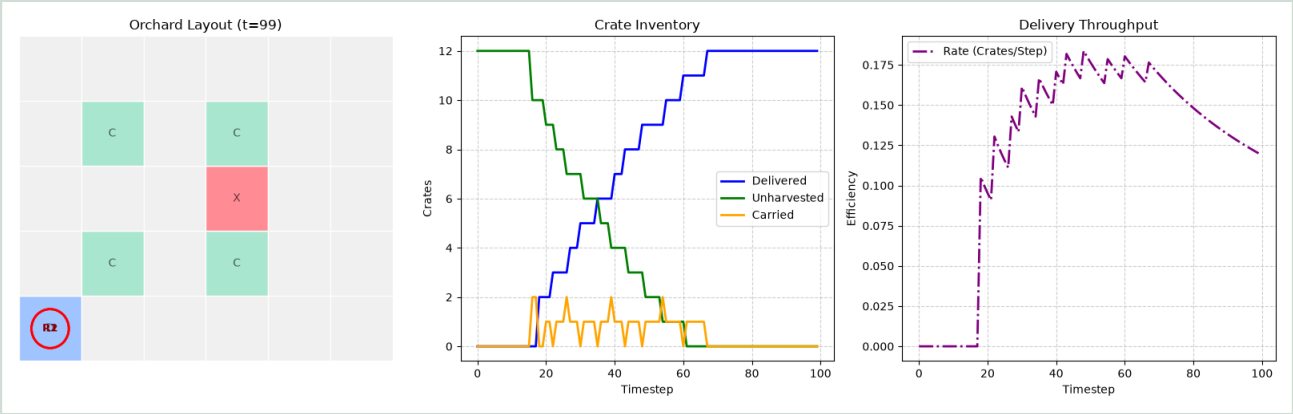


Figure 6. Results of scenario 3 simulation showing the crate movement over time, the efficiency of the model and the final layout.

Software Robustness

In accordance with the rigorous software quality and failure prevention standards of COMP5005 (COMP5005 2026, Lecture 8), a series of validation, topological tests and tests of edge cases were designed. If external data is provided that is invalid, a robust software system should not crash at runtime nor should it silently pass on errors. Smart Orchard WA can detect irregularities instead of relying on simple exceptions in pre-execution stages.

The tests in the `test/` directory were used to test the system's error handling and edge-case handling by intentionally inputting test datasets:

```
PS C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA> python smartfarm.py -f invalide_map.csv -p params_s1.csv
Traceback (most recent call last):
  File "C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA\smartfarm.py", line 190, in <module>
    main()
    ~~~~^^
  File "C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA\smartfarm.py", line 180, in main
    run_batch(map_filename, parameter_filename)
    ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA\smartfarm.py", line 154, in run_batch
    farm.validate(robots)
    ~~~~~~~~~~~~~^^^^^^^^
  File "C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA\farm.py", line 181, in validate
    raise ValueError(
    ...<2 lines>...
    )
ValueError: Validation failed: Crop block at (2, 2) has no adjacent driveable track.
PS C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA>
```

Fig 7. Image showing the exception handling when given an invalid map.

```
PS C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA> python smartfarm.py -f map_s1.csv -p invalid_params.csv
Batch Validation Error: Error: robot_count must be positive.
PS C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA>

    ~~~~^^
  File "C:\Users\45552\Datos\Cursos\Curtin University\COMP5005 Fundamentals of Programming\Assignment\smartOrchardWA\smartfarm.py", line 180, in main
    run_batch(map_filename, parameter_filename)
```

Fig 8. Error message given when we try to run the batch mode with a negative robot count (-2).

Conclusion

The Smart Orchard WA simulation is a successful meeting of all the specifications. The system is highly tolerant of invalid configurations, mathematically correct (checked by Conservation Law), and algorithmically efficient (based on Manhattan heuristic pruning). Separating the simulation engine and the data analysis pipeline makes it highly maintainable.

Future Work

1. **Dynamic Collision Avoidance:** Improve the routing algorithm to avoid having two robots in the same track cell at the same time, instead of the current static pathing.
2. **Battery Management:** Adding more complex heuristic decision making, since agents must be sent back to depots for recharging, due to constraints on power.

References

COMP5005. “Fundamentals of Programming: Lecture 6.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 7.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 8.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 9.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 10.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 11.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Lecture 12.” Accessed Oct 13th, 2026 via Blackboard

COMP5005. “Fundamentals of Programming: Practical 5, Activity 2.” Accessed Oct 13th, 202 via Blackboard

Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms*. 4th ed. Cambridge, MA: MIT Press.

Curtin University. 2026. Chicago 17th Author-Date Referencing Guide. UniSkills. https://uniskills.library.curtin.edu.au/referencing/chicago17/introduction/

Foley, James D., Andries van Dam, Steven K. Feiner, and John F. Hughes. 2013. *Computer Graphics: Principles and Practice*. 3rd ed. Boston, MA: Addison-Wesley.

Python Software Foundation. 2026. Python 3.14.6 Documentation: Collections, CSV, and OS Modules. https://docs.python.org/3/

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson.